MARMARA UNIVERSITY

Electrical & Electronics Engineering Dept.

CSE 1145
Introduction to
Computer Programming

PROJECT REPORT

Group Number: 27

	Name Surname	**Contribution Details**
A	Mehmet Ali Kamalak	Question 1 and 2, Preparing Report
B	Rumeysa Şenol	Question 2 and 3, Ques4 Flow Chart, Preparing Report
C	Fatime Emin	Question 5 and 2, Ques4 Flow Chart, Preparing Report
D	Helia Rahimi	Question 4, Preparing Report

Fall 2024-2025

Table of Contents

Part #1 5

1.1 Brief explanation of the problem 5

1.2 Flow Chart of the Program 5

1.3 Variable definition List 5

1.4 Function definition List 5

1.5 Included Libraries 6

1.6 Screenshots of the sequence 6

1.7 Extra features 7

Part #2 7

2.1 Brief explanation of the problem 7

2.2 Flow Chart of the Program 8

2.3 Variable definition List 8

2.4 Function definition List 8

2.5 Included Libraries 8

2.6 Screenshots of the sequence 9

2.7 Extra features 9

Part #3 9

3.1 Brief explanation of the problem 9

3.2 Flow Chart of the Program 10

3.3 Variable definition List 10

3.4 Function definition List 10

3.5 Included Libraries 11

3.6 Screenshots of the sequence 11

3.7 Extra features 11

Part #4 11

4.1 Brief explanation of the problem 11

4.2 Flow Chart of the Program 12

4.3 Variable definition List 12

4.4 Function definition List 12

4.5 Included Libraries 13

4.6 Screenshots of the sequence 13

4.7 Extra features 14

Part #5 14

5.1 Brief explanation of the problem 14

5.2 Flow Chart of the Program 14

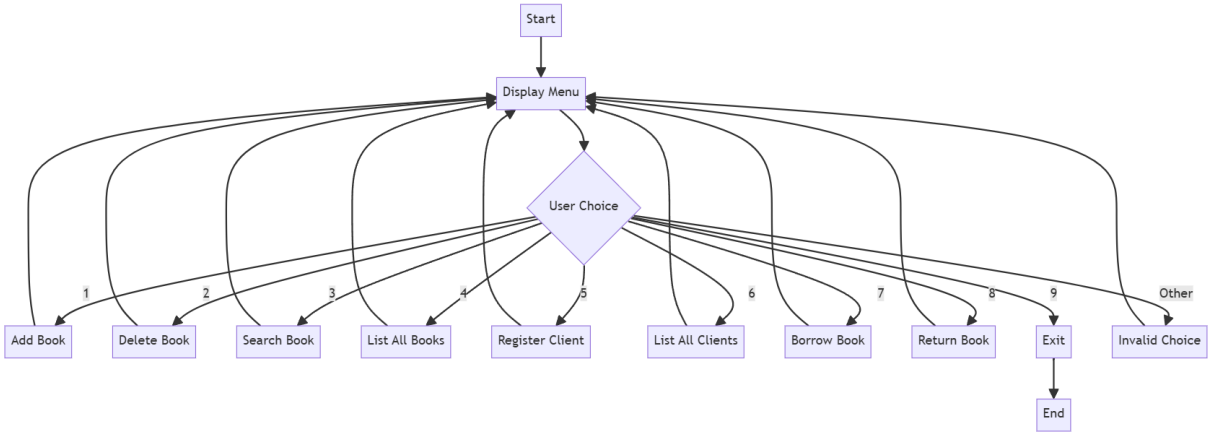
5.3 Variable definition List 14
5.4 Function definition List 15
5.5 Included Libraries 15
5.6 Screenshots of the sequence 15
5.7 Extra features 16
References 16

Part #1

1.1 Brief explanation of the problem

This program is a library management system designed to manage books and customers. It allows users to perform actions such as adding, deleting, searching for books, registering customers, borrowing, and returning books.

1.2 Flow Chart of the Program



1.3 Variable definition List

#	Type	Name	Comment
1	Book	books	Array used to store all book information.
2	customer	customers	Array used to store all customer information.
3	int	numberofbook	Total number of added books.
4	int	numberofcustomer	Total number of registered customers.
5	int	choice	Stores the user selection from the main menu.
6	int	bookID	Used to store the the book ID for add, delete or search operations.
7	int	customerID	Used to for operations involving customer data.
8	char	query	Stores the search query.
9	int	bookfound	Holds the index of the found book in search operations.
10	int	found	Used the check if a specific record exists

1.4 Function definition List

#	Return Type	Name	Parameters	Comment

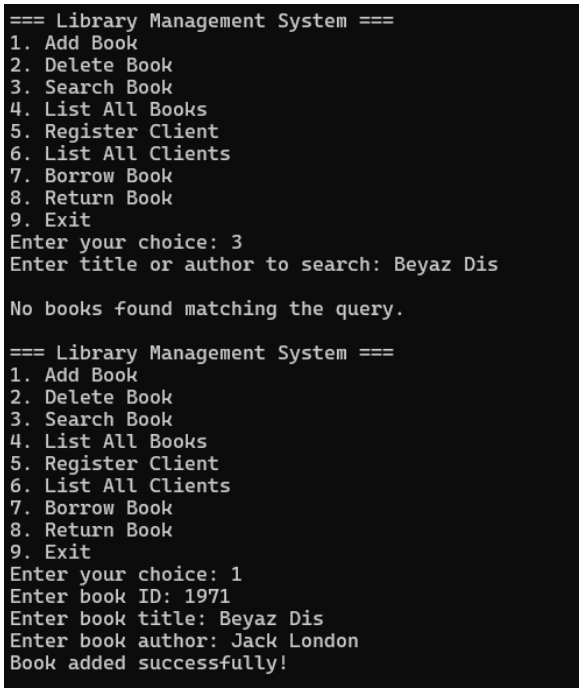
1	void	addbook	None	Adds a new book to the ‘books’ array anad increments the book count.
2	void	listbook	None	Lists all books in library with their details.
3	void	deletebook	None	Deletes a book based on the ID provided by the user.
4	void	searchbook	None	Searches for books by title or author.
5	void	registercustomer	None	Register a new customer to in ‘customers’
6	void	listcustomer	None	Lists all registered customers with their details.
7	void	borrowbook	None	Customer to borrow a book by specifying their ID and book ID.
8	void	returnbook	None	Customer to return a borrowed book by specifying the book ID.
9	void	displaymenu	None	Displays the main menu options to the user.

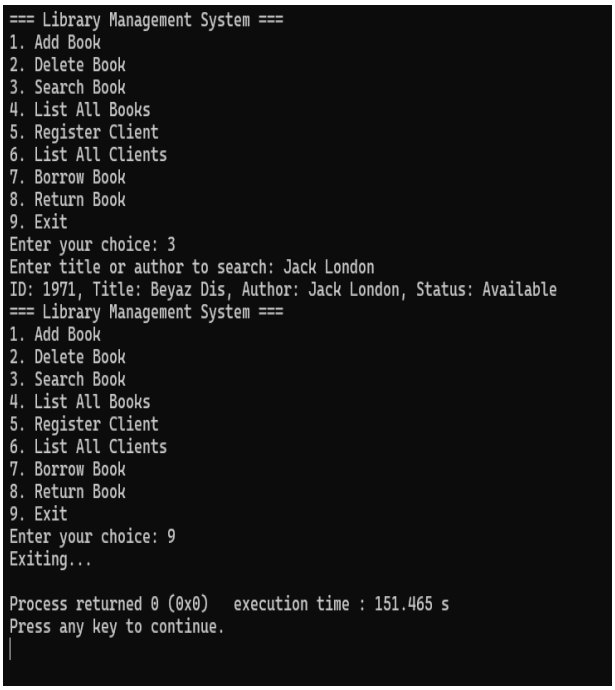
1.5 Included Libraries

#	Return Type	Name	Parameters	Comment
1		<stdio.h>	None	Used for input and output functions.(printf,scanf...)
2		<stdlib.h>	None	General utility functions
3		<string.h>	None	Used for string manipulation functions (strstr, strcspn)

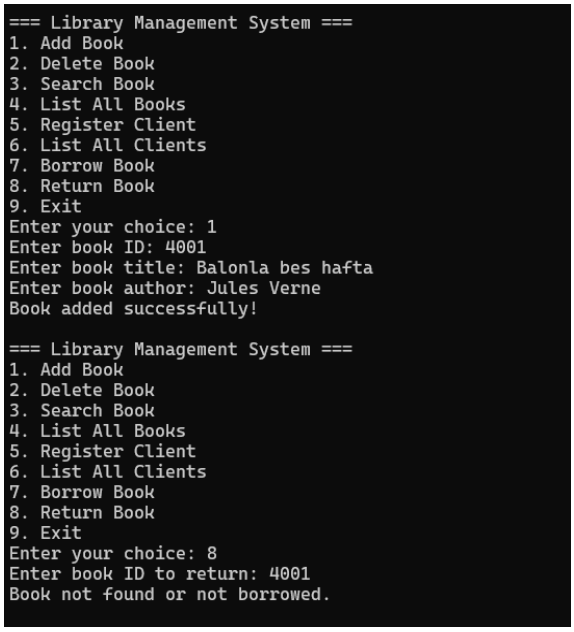
1.6 Screenshots of the sequence

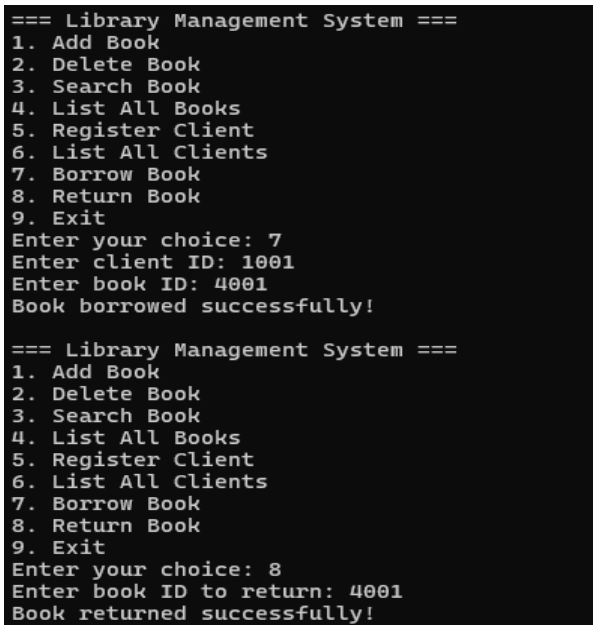
Firstly, we selected 3. (Search Book) and asking the book we wanted to search. But it was not found. Then we added the book to the library and searched again and displayed the name of the book, ID, author and borrowing status.





Secondly, we will try to return the book (9/ Return Book). To do this, we first need to add the book to our library. Then it needs to be borrowed. Finally, we return the book again.





1.7 Extra features

There is no extra feature.

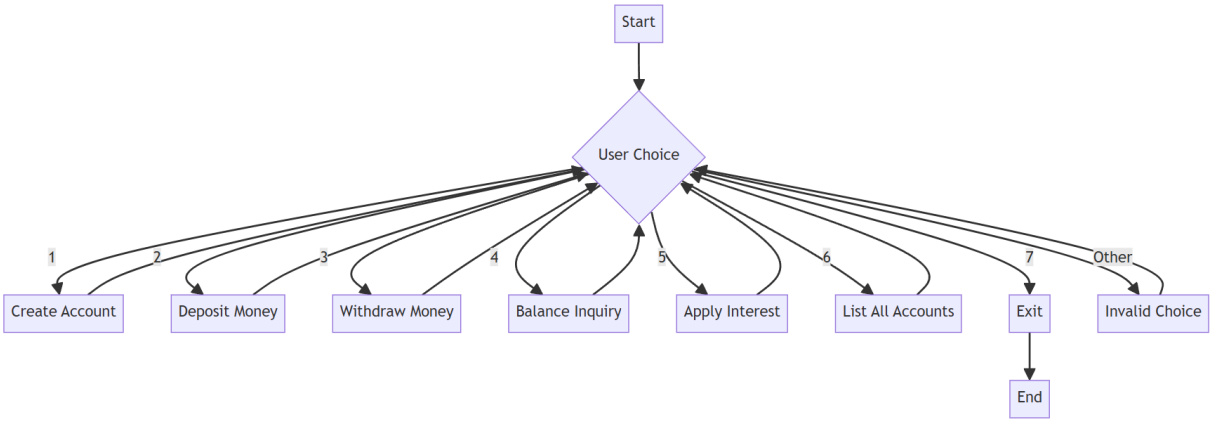
Part #2

2.1 Brief explanation of the problem

This program simulates a simple banking system that allows users to create accounts, deposit money, inquire about balances, apply interest to savings accounts, and list all accounts. It uses a structure to

store account details and provides a menu-driven interface for interaction. The program also ensures basic security through PIN verification.

2.2 Flow Chart of the Program



2.3 Variable definition List

#	Type	Name	Comment
1	Account[]	accounts	Array to store all the account information for the banking system.
2	int	accountCount	Tracks the number of accounts currently in the system.
3	Int	nextAccountNumber	Stores the next available account number for new accounts.

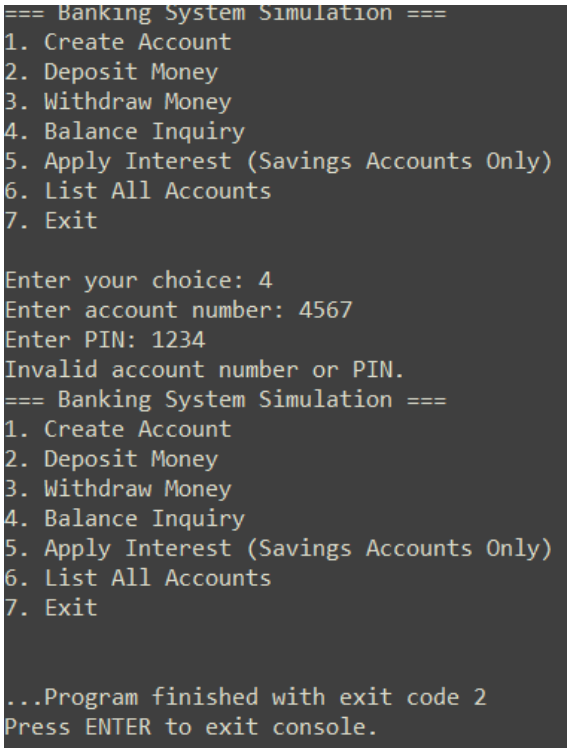
2.4 Function definition List

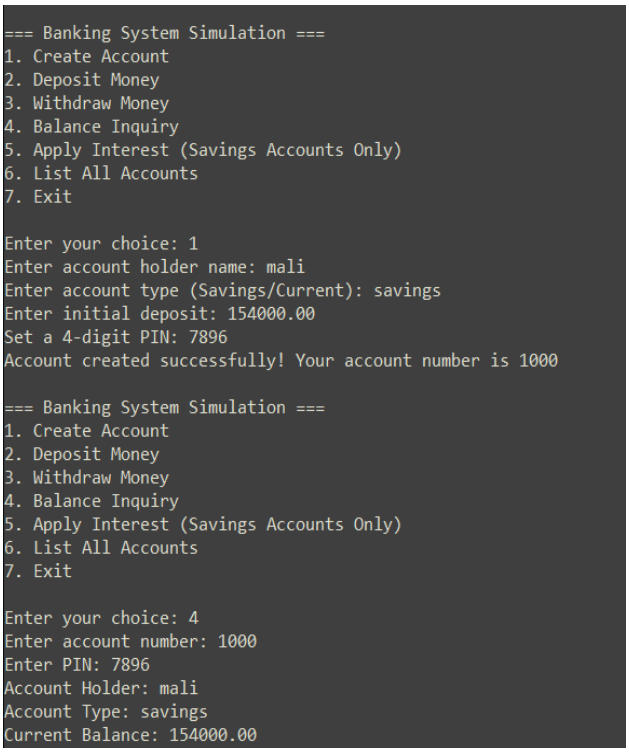
#	Return Type	Name	Parameters	Comment
1	void	openAccount	None	Creates a new account and initializes its details.
2	void	addFunds	None	Allows a user to deposit money into an existing account.
3	void	checkBalance	None	Displays the account details and balance for a specific account.
4	void	calculateInterest	None	Applies interest to all savings accounts in the system.
5	void	displayAccounts	None	Lists all accounts and their details in the system.

2.5 Included Libraries

#	Return Type	Name	Parameters	Comment
1		<stdio.h>	None	Used for input and output functions.(printf,scanf...)
2		<string.h>	None	Used for string manipulation functions (strstr, strcspn)

2.6 Screenshots of the sequence





2.7 Extra features

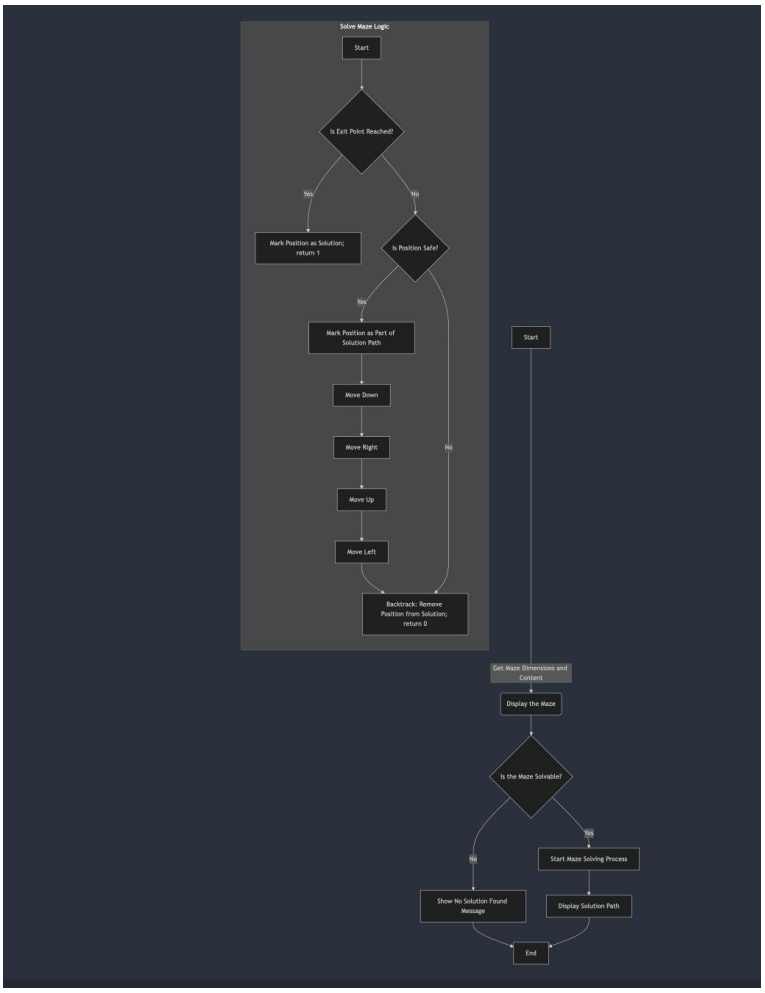
There is no extra feature.

Part #3

3.1 Brief explanation of the problem

The problem involves designing a maze-solving program in C that uses a 2D array to represent a maze and applies the Depth-First Search (DFS) algorithm to find a path from the start point to the end point. The program visualizes the maze, marks the solution path with *, and indicates if no path exists.

3.2 Flow Chart of the Program



3.3 Variable definition List

#	Type	Name	Comment
1	int	maze[MAX] [MAX]	A 2D array representing the input maze where 0 is an open path and 1 is a wall.
2	Int	solution[MAX][MAX]	A 2D array for marking the solution path during the DFS process.
3	int	rows	Stores the number of rows in the maze.
4	Int	cols	Stores the number of columns in the maze.
5	int	x,y	Used for the corret coordinates during DFS traversal.

3.4 Function definition List

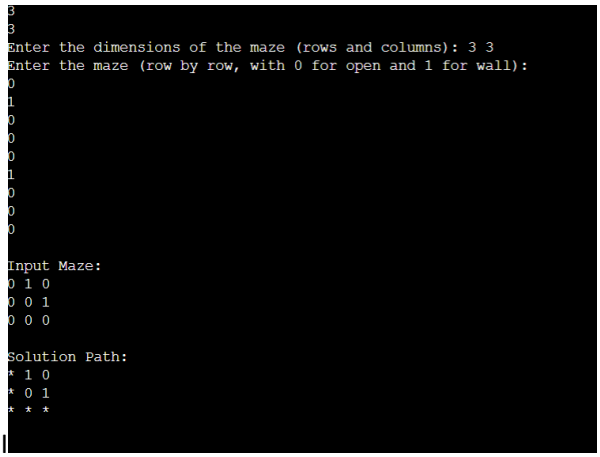
#	Return Type	Name	Parameters	Comment
1	void	acceptMaze	void	Collects the maze dimensions and contents from the user.
2	void	printMaze	Int maze[Max][MAX], int r, int c	Displays the input or solved maze to the console.

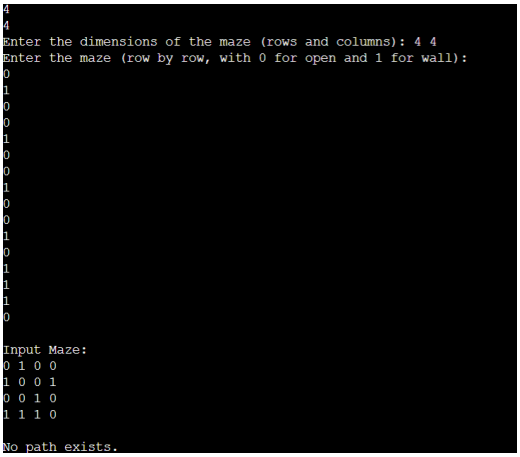
3	int	isSafe	int x, int y	Checks if a cell is within bounds, open, and unvisited.
4	int	solveMaze	int x, int y	Applies DFS algorithm to find the solution path.

3.5 Included Libraries

#	Library	Comment
1	#include <stdio.h>	Provides input/output functions like printf and scanf.

3.6 Screenshots of the sequence





3.7 Extra features

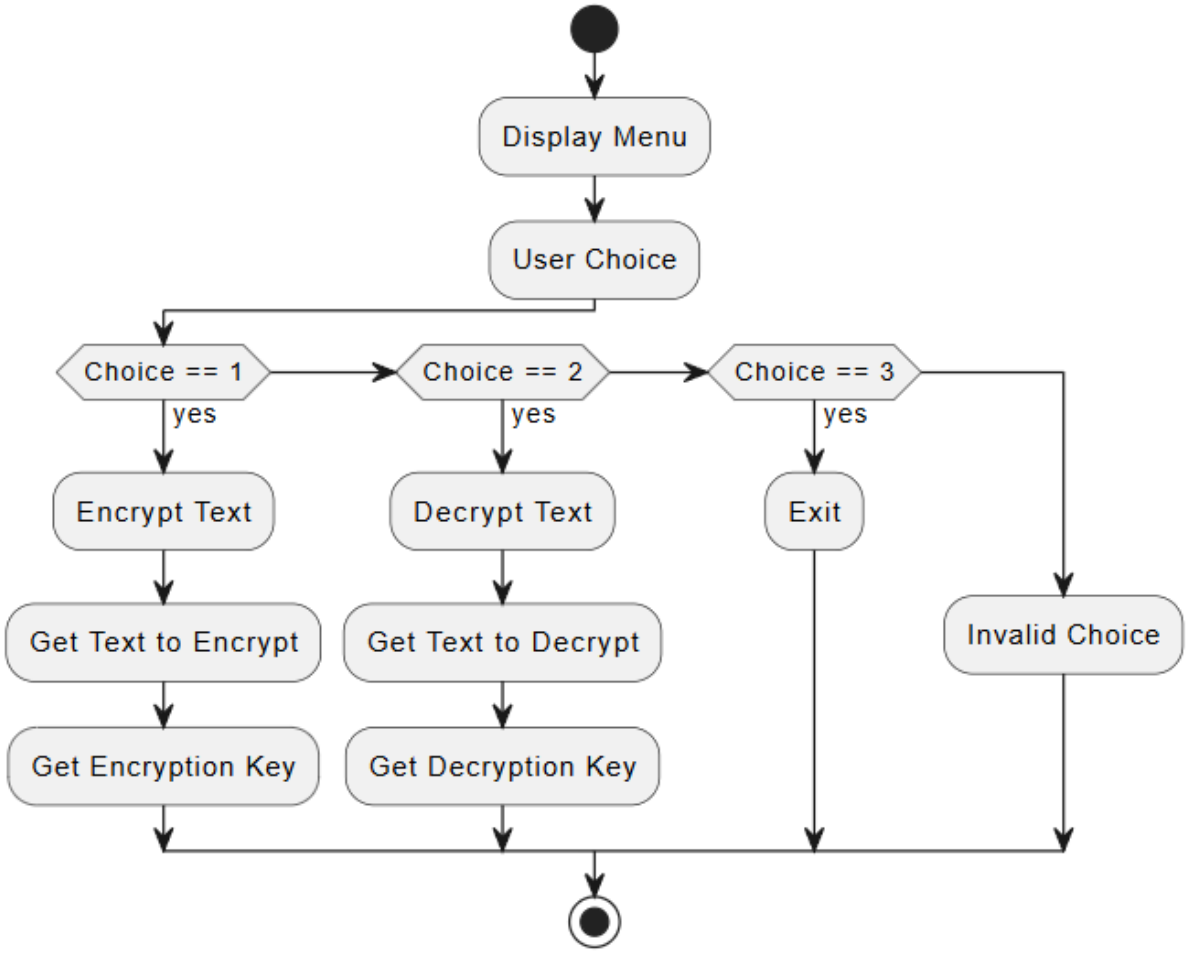
There is no extra feature.

Part #4

4.1 Brief explanation of the problem

This part of the program requires the user to input text for encryption or decryption using a Caesar Cipher. The program displays the input text before applying the specified shift using a key.

4.2 Flow Chart of the Program



4.3 Variable definition List

#	Type	Name	Comment
1	char	text	Stores the input text (either for encryption or decryption). It holds the user input string.
2	int	key	Stores the key (integer) for the encryption or decryption operation. It determines how many positions to shift the characters in the ASCII table.
3	int	choice	stores the user's menu choice (1 for encryption, 2 for decryption, 3 to exit the program).
4	int	i	A loop counter used to iterate through each character of the text string in both the encryptText and decryptText functions.

4.4 Function definition List

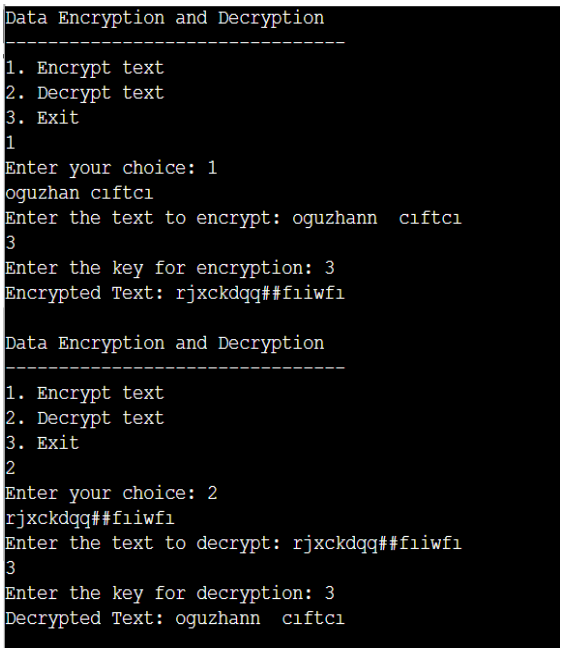
#	Return Type	Name	Parameters	Comment
1	void	encryptText	Char * text, int key	This function encrypts the given text using the Caesar cipher. It shifts each character in the text by the specified key value, ensuring the shift stays within the printable ASCII range.

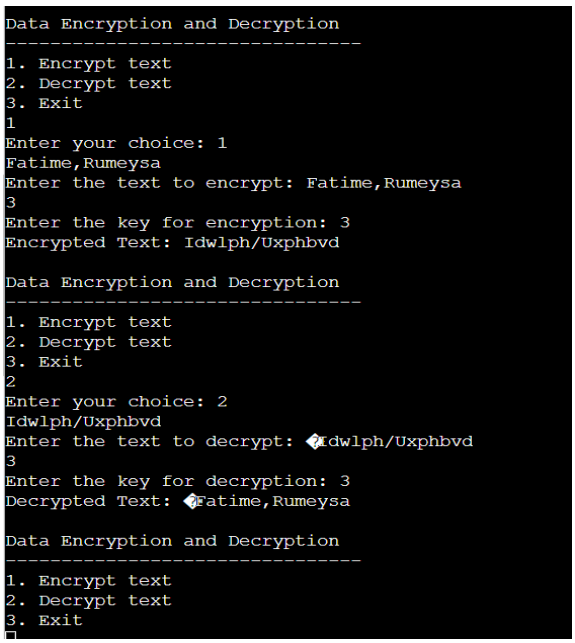
2	void	descryptText	Char * text, int key	This function decrypts the given text by reversing the encryption process. It shifts each character in the text by the negative of the specified key, restoring the original characters.
3	int	main	None	This is the main entry point of the program. It presents the menu to the user, handles user input, and calls either the encryptText or decryptText function based on the user's choice. It also handles the flow of the program (looping until the user exits).

4.5 Included Libraries

#	Return Type	Name	Parameters	Comment
1	Header file	#include <stdio.h>	None	Provides functionality for input and output operations (e.g., `printf`, `scanf`, `fgets`). Used for displaying text and reading user input.
2	Header file	#include <string.h>	None	Provides string manipulation functions (e.g., `strcspn`, `fgets`). Used for handling and processing strings.

4.6 Screenshots of the sequence





4.7 Extra features

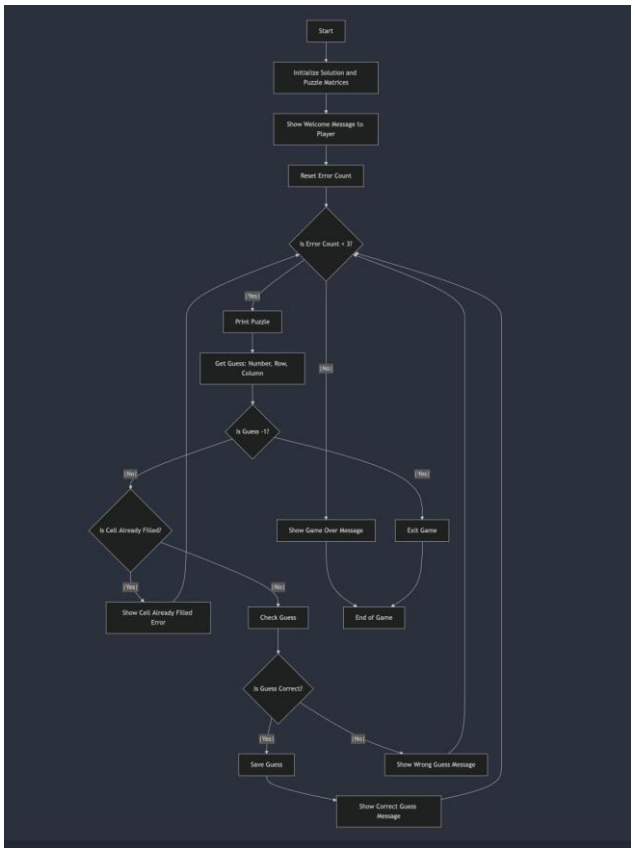
There is no extra feature.

Part #5

5.1 Brief explanation of the problem

This program implements a simple **Sudoku game** where the player attempts to fill in the missing numbers in a 9x9 grid. The game allows a maximum of 3 incorrect guesses. The player provides a number and its row and column position, and the program checks the validity of the move against the solution grid. The game ends when the player either solves the puzzle or exceeds the maximum number of errors.

5.2 Flow Chart of the Program



5.3 Variable definition List

#	Type	Name	Comment
1	Global-integer	SİZE	This variable defined as a global variable with value of 9. It is used to identify the grids - the rows and columns of the sudoku. By this way a 2D sudoku is designed.
2	Global-integer	MAX_ERRORS	It is determined as a global variable with value of 3. It is used to specify maximum limitation of mistakes while player is playing the game. The game ends when the number of incorrect inputs reach to MAX_ERRORS.
3	Local-integer	errors	Tracks the number of incorrect guesses made by the player.

4	Local-integer	row, col, num	Represents the row, column, and number input by the player.
5	Local-integer	isComplete	A flag used to check if the puzzle is completely solved (1 if solved, 0 otherwise).
6	2D-array integer	puzzle	Holds the solution grid to verify the correctness of the player's inputs.
7	2D-array integer	solution	Holds the solution grid to verify the correctness of the player's inputs.

5.4 Function definition List

#	Return Type	Name	Parameters	Comment
1	void	printfSudoku	int grid[SIZE][SIZE]	Prints the current state of the Sudoku grid to the console.
2	void	playSudoku	int puzzle[SIZE][SIZE], int solution[SIZE][SIZE]	Contains the main game logic, including user input, validation, and game termination checks.
3	int	main	none	Initializes the puzzle and solution grids, welcomes the player, and starts the game loop.

5.5 Included Libraries

#	Return Type	Name
1	stdio.h	Provides functionality for input/output operations such as printf and scanf.
2	stdlib.h	Includes utility functions such as exit, although it is not directly used in the current implementation.

5.6 Screenshots of the sequence

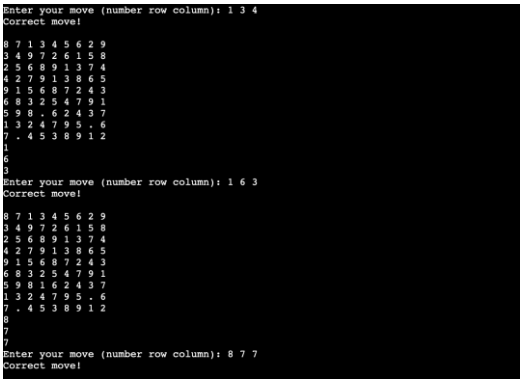
Here is the example of a 9x9 sudoku puzzle:

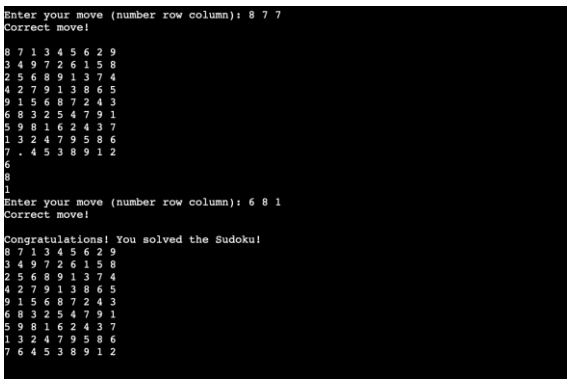
8	7	1	3	4	5	6	2	9
3	4	9	7	2	6	1	5	8
2	5	6	8	9	1	3	7	4
4	2	7	9	1	3	8	6	5
9	1	5	6	8	7	2	4	3
6	8	3	2	5	4	7	9	1
5	9	8	1	6	2	4	3	7
1	3	2	4	7	9	5	8	6
7	6	4	5	3	8	9	1	2

```
Welcome to Sudoku! Fill in the grid step by step.
Enter a number followed by row and column
(e.g., 5 3 8 to place 5 at row 3, column 8).
You have a maximum of 3 incorrect guesses. Type -1 to exit.

8 7 1 3 4 5 6 2 9
3 4 . 7 2 6 1 5 8
2 5 6 8 9 1 3 7 4
4 2 7 9 . 3 8 6 5
9 1 5 6 8 7 2 4 3
6 8 3 2 5 4 7 9 1
5 9 8 . 6 2 4 3 7
1 3 2 4 7 9 5 . 6
7 . 4 5 3 8 9 1 2
9
1
2
Enter your move (number row column): 9 1 2
Correct move!

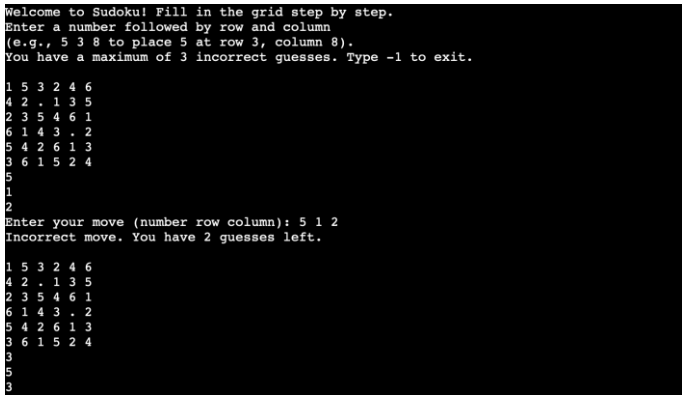
8 7 1 3 4 5 6 2 9
3 4 9 7 2 6 1 5 8
2 5 6 8 9 1 3 7 4
4 2 7 9 . 3 8 6 5
9 1 5 6 8 7 2 4 3
6 8 3 2 5 4 7 9 1
5 9 8 . 6 2 4 3 7
1 3 2 4 7 9 5 . 6
7 . 4 5 3 8 9 1 2
1
3
4
Enter your move (number row column): 1 3 4
Correct move!
```

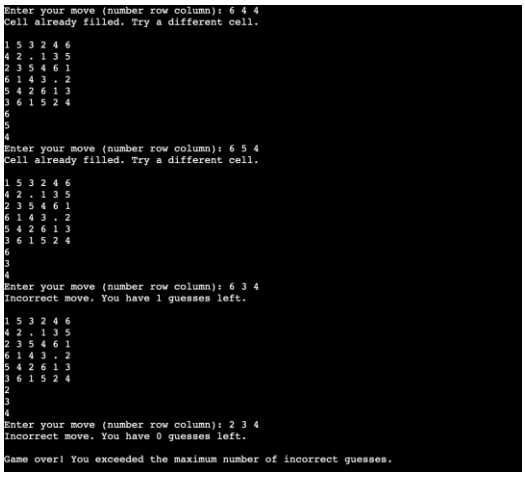




Here is the example of a 6x6 sudoku puzzle:

1	5	3	2	4	6
4	2	6	1	3	5
2	3	5	4	6	1
6	1	4	3	5	2
5	4	2	6	1	3
3	6	1	5	2	4





5.7 Extra features

- This code has the ability to solve any sudoku puzzle defined in different dimensions.

References